



Probabilistic Machine Learning

21. Clustering

김무진 / '26.05.21

22. Recommender Systems



Computational Data Science LAB



CONTENTS

1. Clustering
 2. Algorithm
 3. Spectral Clustering
 4. Bi-Clustering
 5. Recommender Systems
- 
- 

01 | Clustering

Introduction

- Clustering
 - ✓ 군집화는 매우 일반적인 비교사 학습 방식이며, 두 가지 주된 종류가 있음
 - 입력이 데이터 표본 $D = \{x_n: n = 1:N\}$ 의 집합인 경우
 - 입력이 $N \times N$ 쌍별 비유사도 인 경우
 - ✓ 군집화 알고리즘의 질을 평가하는 것은 어려운 문제

- Purity

- ✓ $\text{Purity} \triangleq \sum_i \frac{N_i}{N} p_i$

- ✓ $\frac{6}{17} \frac{5}{6} + \frac{6}{17} \frac{4}{6} + \frac{5}{17} \frac{3}{5} = \frac{5+4+3}{17} = 0.71$



Figure 21.1: Three clusters with labeled objects inside.

01 | Clustering

Rand Index

- Rand Index

- ✓ $U = \{u_1, \dots, u_R\}, V = \{v_1, \dots, v_C\}$

- ✓ $R \triangleq \frac{TP+TN}{TP+FP+FN+TN}$

- ✓ TP : 예측에서도 같은 군집이고, 정답 라벨도 같은 쌍

- ✓ TN : 예측에서는 같은 군집이지만, 정답 라벨은 다른 쌍

- ✓ FN : 정답 라벨은 같은데, 예측에서는 다른 군집으로 갈라진 쌍

- ✓ FP : 예측에서는 같은 군집인데, 정답 라벨은 다른 쌍

- Purity & Rand Index

- ✓ 각 군집이 얼마나 한 라벨로 깔끔하게 모였는지 → Purity

- ✓ 두 데이터가 같은 그룹인지/다른 그룹인지의 관계를 전체적으로 잘 맞췄는지 → Rand Index



Figure 21.1: Three clusters with labeled objects inside.

- ✓ $TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 40$

- ✓ $TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 20$

- ✓ $FN = \binom{8}{2} + \binom{5}{2} + \binom{4}{2} - TP = 24$

- ✓ $\frac{20+72}{20+20+24+72} = 0.68$

02 | Algorithm

K-Means++ Algorithm

- 초기화 방식

- ✓ 데이터 중 K 개를 랜덤으로 뽑고, 초기 중심 M_K 로 사용하는 방식 → 초기 랜덤 중심에 따라 한계 존재
- ✓ 하나씩 순서대로 뽑으면서, 이미 뽑힌 중심점들과 멀리 떨어진 점일수록 다음 중심점으로 뽑힐 확률을 높임 → K-Means ++

- K-Means++ Algorithm

- ✓ 첫 번째 중심은 랜덤 뽑음
- ✓ 각 데이터가 현재 중심에서 얼마나 먼지 계산
- ✓ 멀리 있는 점일 수록 다음 중심으로 뽑힐 확률을 높임
- ✓ 중심이 K 개가 될 때까지 반복 후 , K-Means 시작

- 수식

- ✓ $D_{t-1}(\mathbf{x}) = \min_{k=1}^{t-1} \|\mathbf{x} - \boldsymbol{\mu}_k\|_2^2$

- ✓ $P(\mu_t = x_n) = \frac{D_{t-1}(x_n)}{\sum_{n'=1}^N D_{t-1}(x_{n'})}$

02 | Algorithm

K-Medoids Algorithm

- K-Medoids Algorithm

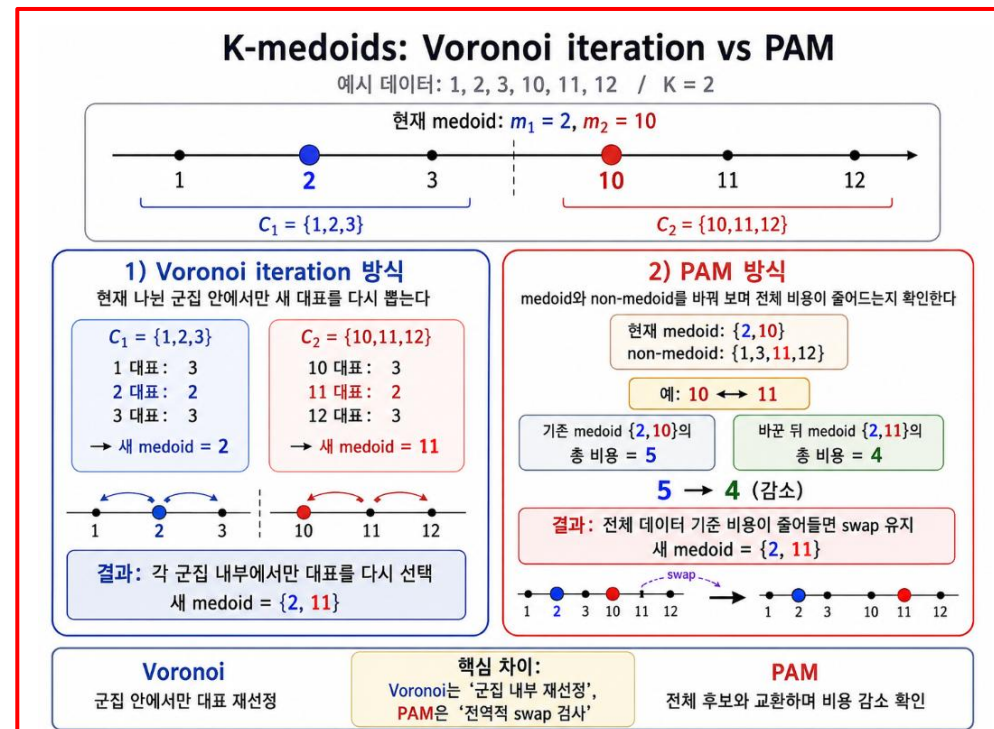
- ✓ K-means의 중심은 평균이기 때문에 이상치에 약함
- ✓ K-medoids는 군집 안에서 다른 점들과의 평균 거리 또는 거리 합이 가장 작은 **실제 데이터**를 대표점으로 고름 → medoid
- ✓ 중요한 것은, 평균이 잘 정의되지 않을 수도 있는(feature vector가 아닌 경우) $\mathbb{R}^D$ 이 아닌 데이터에 적용할 수 있음
- ✓ K-medoids 알고리즘의 입력은 $N \times D$ 특성 행렬이 아닌 $N \times N$ 쌍별 거리 행렬

- Voronoi Iteration Algorithm

- ✓ 각 데이터를 가까운 medoid에 배정
- ✓ 각 군집 안에서 대표 medoid를 다시 선택
- ✓ 비용 계산 후 과정 반복

- PAM(Partitioning Around Medoids) Algorithm

- ✓ 초기 medoid 선택
- ✓ 모든 데이터를 가장 가까운 medoid에 배정
- ✓ medoid → non-medoid Swap후, 각 데이터가 가장 가까운 medoid까지 가는 거리 합이 감소하면 유지



03 | Spectral Clustering

Spectral Clustering

- Background

- ✓ 점들의 좌표 모양을 직접 보지 않고, 점들끼리 얼마나 강하게 연결되어 있는지를 보게 됨
- ✓ 데이터를 그래프로 만든 후, 간선의 강도는 유사도의 측정치
- ✓ 목표는 유사한 점의 k개 군집을 찾는 것

- 수식

- ✓ $cut(S_1, \dots, S_k) \triangleq \frac{1}{2} \sum_{k=1}^K W(S_k, \overline{S_k})$ 즉, 군집을 나누기 위해 끊어야 하는 연결의 총 강도
- ✓ 단순히 cut 만 최소화하는 최적해는 단지 하나의 노드만 나머지로부터 분할하는 것인 경우가 많음 → 정규화 필요
- ✓ $Ncut(S_1, \dots, S_k) \triangleq \frac{1}{2} \sum_{k=1}^K \frac{cut(S_k, \overline{S_k})}{vol(S_k)}$, $vol(S_k)$ 는 군집 S_k 가 가진 전체 연결량
- ✓ S_k 가 가진 전체 연결량 중에서 바깥으로 나간 연결의 비율을 보게 됨으로써 더 합리적인 목적함수를 가짐

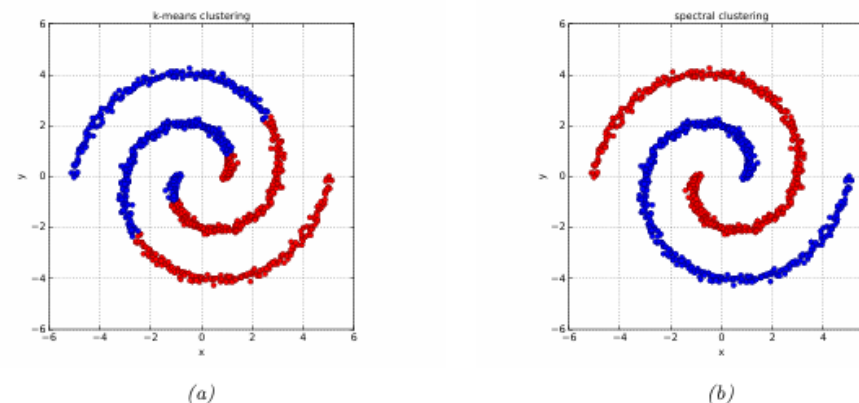


Figure 21.19: Results of clustering some data. (a) K-means. (b) Spectral clustering. Generated by [spectral_clustering_demo.ipynb](#).

03 | Spectral Clustering

Eigenvectors of the graph Laplacian encode the clustering

- NP-hard

- ✓ Ncut의 경우 목적함수를 최소화하는 이항 벡터 $c_i \in \{0,1\}^N$ 을 찾는 측면으로 형식화 가능
- ✓ N이 많이 커진다면 ?? → 무수히 많은 조합에 따른 NP-hard → $c_i \in \mathbb{R}^N$ 으로 완화하여 실수 값 허용

- Laplacian matrix

- ✓ $L = D - W$, $D = \text{diag}(d_i)$, W 는 유사도 행렬
- ✓ $L_1 = [2, -1, 0, 0, -1, 0]$, $(Lf_1) = 2f_1 - f_2 - f_5 = (f_1 - f_2) + (f_1 - f_5)$
- ✓ $Lf = \lambda f$, λ 가 작으면 Lf 도 작아지게 됨. 즉 L 이 f 에 크게 반응하지 않음
- ✓ $K = 2$ 라면, 부호나 threshold로 분리 가능
- ✓ 일반적으로 K 개의 고유벡터를 모아서 $U = [u_1, u_2, \dots, u_K]$ 각 row를 원래 데이터의 새 feature로 본 후 K-Means 수행

6-Element Binary Vector for Spectral Clustering

Vector Notation

$$f = [0, 0, 0, 1, 1, 1]^T$$

Column Vector Form

$$f = \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \end{matrix} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \end{pmatrix} \left\{ \begin{array}{l} \text{Cluster 1} \\ \text{(nodes 1-3)} \\ \text{Cluster 2} \\ \text{(nodes 4-6)} \end{array} \right.$$

Labelled graph	Degree matrix	Adjacency matrix	Laplacian matrix
	$\begin{pmatrix} 2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{pmatrix}$

Figure 20.39: Illustration of the Laplacian matrix derived from an undirected graph. From https://en.wikipedia.org/wiki/Laplacian_matrix. Used with kind permission of Wikipedia author AzaToth.

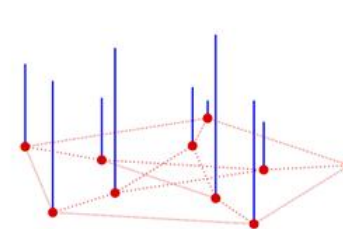


Figure 20.40: Illustration of a (positive) function defined on a graph. From Figure 1 of [Shu+13]. Used with kind permission of Pascal Frossard.

04 | Bi-clustering

Bi-clustering

- Bi-clustering

- ✓ 일반 clustering은 보통 데이터 행만 묶음
- ✓ Bi-clustering은 행과 열을 동시에 군집화 하는 방법
- ✓ 데이터 행렬 한 칸 X_{ij} 는 i 번째 동물이 j 번째 특징을 갖는 지 여부

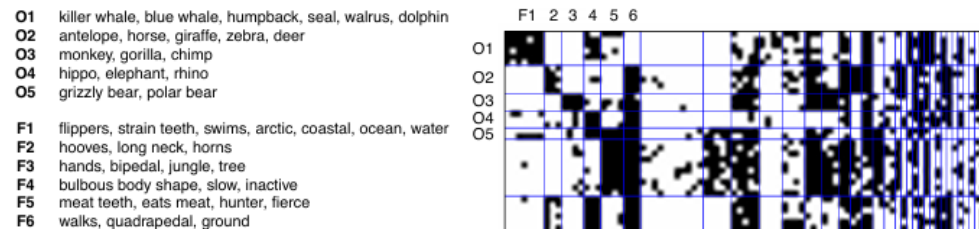


Figure 21.20: Illustration of biclustering. We show 5 of the 12 organism clusters, and 6 of the 33 feature clusters. The original data matrix is shown, partitioned according to the discovered clusters. From Figure 3 of [Kem+06]. Used with kind permission of Charles Kemp.

- Probabilistic Generative Model

- ✓ $P(U) = \prod_{i=1}^{N_r} \text{Unif}(u_i | \{1, \dots, N_u\})$
- ✓ $P(V) = \prod_{j=1}^{N_c} \text{Cat}(v_j | \{1, \dots, N_v\})$
- ✓ $P(X|U, V, \theta) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} P(X_{ij} | \theta_{u_i, v_j})$
- ✓ W 즉, 우리가 관측한 데이터 X 를 가장 그럴듯하게 설명하는 row 군집 U , column 군집 V , 파라미터 θ 를 찾기
- ✓ 이러한 Basic bi-clustering에서는 각 row가 하나의 row clustering에 속함
→ 하나의 row cluster로는 여러 기준의 유사성 동시 설명 불가

- Example

- ✓ {고래, 박쥐, 상어, 독수리}
- ✓ 젓 먹임 관점 → {고래, 박쥐}
- ✓ 물에 사는 특징 → {고래, 상어}
- ✓ column에 따른 다른 군집화 필요

04 | Bi-clustering

Cross cat

- Cross cat

- ✓ Column들을 먼저 여러 feature group으로 나누고, 각 column group마다 row clustering을 따로 하는 방식
- ✓ 열을 먼저 변수 그룹으로 군집화 → 각 변수 그룹마다 행을 따로 군집화 → 각 칸이 속한 블록의 확률분포로 데이터가 생성된다고 보는 모델
- ✓ $P(V) = \prod_{j=1}^{N_c} \text{Unif}(v_j | \{1, \dots, N_v\}) \rightarrow$ column j 가 어떤 column group에 속하는가
- ✓ $P(U) = \prod_{i=1}^{N_r} \prod_{l=1}^{N_v} \text{Unif}(u_{il} | \{1, \dots, N_u\}) \rightarrow$ column group l 을 볼 때, row i 가 어떤 row cluster에 속하는가
- ✓ $P(Z|U, V) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} \mathbb{I}(Z_{ij} = (u_{i,v_j}, v_j)) \rightarrow$ 각 칸 X_{ij} 가 어떤 블록에 속하는지 결정
- ✓ $P(X|Z, \theta) = \prod_{i=1}^{N_r} \prod_{j=1}^{N_c} p(X_{ij} | \theta_{Z_{ij}}) \rightarrow$ 실제 데이터 값 X_{ij} 가 생성되는 부분

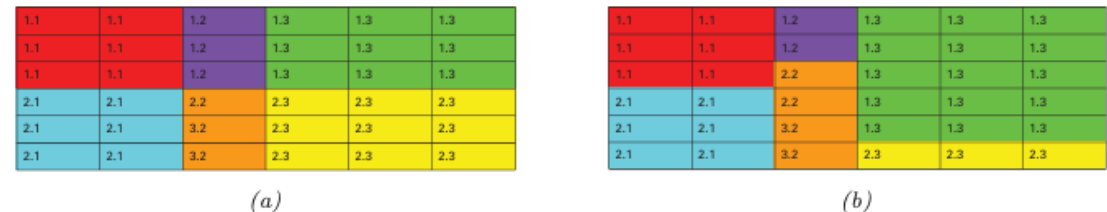


Figure 21.21: (a) Example of biclustering. Each row is assigned to a unique cluster, and each column is assigned to a unique cluster. (b) Example of multi-clustering using a nested partition model. The rows can belong to different clusters depending on which subset of column features we are looking at.

05 | Recommender Systems

Explicit feedback

- Recommender System

- ✓ 추천시스템은 과거 시청한 것, 구매 행위를 기반으로 아이টে을 사용자에게 추천하는 시스템
- ✓ 사용자들이 아이টে에 남긴 평점 행렬을 보고, 아직 보지 않은 아이টে에 대해 사용자가 몇 점을 줄 지 예측하는 것이 핵심

- Netflix

- ✓ 2006년에 480,189명의 사용자가 17,770개의 영화에 매긴 100,480,507개 영화 평점(1~5의 스케일)의 데이터 셋을 공개
- ✓ 많은 데이터처럼 보이지만, 이 행렬도 여전히 99% sparse함

- Explicit feedback

- ✓ 사용자가 시스템에 +1또는 -1 아니면 1~5의 점수와 같은 평점 측면에서 명시적인 피드백을 제공하는 것
- ✓ $Y_{ui} \in \mathbb{R}$ 은 사용자 u 가 아이টে i 에 준 평점일 때, M 은 사용자 수, N 이 아이টে 수일 때, $M \times N$ 행렬로 나타낼 수 있음
- ✓ 이 행렬은 매우 sparse하며, 양분화된 그래프로도 볼 수 있음

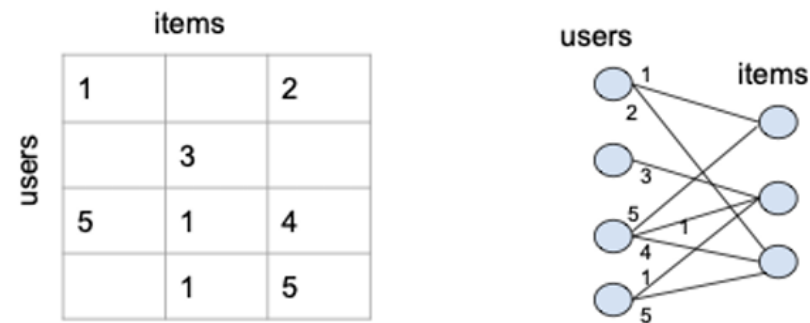


Figure 22.1: Example of a relational dataset represented as a sparse matrix (left) or a sparse bipartite graph (right). Values corresponding to empty cells (missing edges) are unknown. Rows 3 and 4 are similar to each other, indicating that users 3 and 4 might have similar preferences, so we can use the data from user 3 to predict user 4's preferences. However, user 1 seems quite different in their preferences, and seems to give low ratings to all items. For user 2, we have very little observed data, so it is hard to make reliable predictions.

05 | Recommender Systems

Collaborative filtering

- Missing data
 - ✓ User가 item을 아직 못 본 경우
 - ✓ 좋아하지 않을 것 같아서 일부러 안 본 경우
- Collaborative filtering
 - ✓ 사용자가 그들의 평점을 다른 사용자와 공유하여 아이템 추천을 협력한다는 아이디어
 - ✓ 실제로 사용자1이 어떤 영화 i 를 안 봤을 때, 사용자 1과 취향이 비슷한 사용자들이 영화 i 에 대한 평점을 본 후 가중 평균
 - ✓ 수식: $\hat{Y}_{ui} = \sum_{u': Y_{u',i} \neq ?} \text{sim}(u, u') Y_{u',i}$
 - ✓ Sim은 두 사용자가 공통으로 평가한 item을 본 후 비교 → data sparsity문제가 여전히 존재
 - ✓ 따라서 아이템과 사용자를 저 차원 벡터 공간에 배치하는 dense embedding vector를 학습

05 | Recommender Systems

Matrix Factorization

- Matrix Completion

- ✓ 추천 시스템은 Y 의 비어 있는 항목을 예측하는 Matrix Completion(Z)의 하나로 볼 수 있음
- ✓ $\mathcal{L}(Z) = \sum_{ij: Y_{ij} \neq ?} (Z_{ij} - Y_{ij})^2 = \|Z - Y\|_F^2$

- Matrix Factorization

- ✓ 빈칸을 채우는 방법은 너무 많아서 정답이 하나로 정해지지 않음 → 평점행렬(Y)은 Low-Rank라는 가정
- ✓ Low-Rank 가정 : 평점은 몇 개의 숨은 요인(K)로 설명됨
- ✓ $Z = UV^T \approx Y$, Z 는 예측행렬, $U \in \mathbb{R}^{M \times K}$ 사용자 벡터 행렬, $V \in \mathbb{R}^{N \times K}$ 는 아이템 벡터 행렬
- ✓ 즉, 더 큰 예측행렬 Z 자체를 학습하는 것이 아닌, 더 작은 두 개 행렬 U, V 를 학습
- ✓ $\hat{y}_{ui} = u_u^T v_i$ 의 예측 형식

05 | Recommender Systems

Implicit feedback

- Implicit feedback

- ✓ Implicit feedback은 사용자의 행동에서 간접적으로 드러난 신호
- ✓ 예를 들어 사용자 u 가 본 영화의 리스트를 양성으로 다루고, 나머지 모든 영화는 음성으로 간주할 수 있음
- ✓ 또는 영화 i 를 봤지만 영화 j 를 보지 않았다면, 이들이 i 를 j 보다 선호한다는 암묵적인 신호로 볼 수 있음
- ✓ 결과 데이터 $y_n = (u, i, j)$ 형식의 튜플 집합

- Bayesian Personalized Ranking

- ✓ 모델을 (u, i, j) 형식의 데이터에 적합 시키려면 ranking loss를 사용해야 하며, 사용자가 본 아이템 i 가 안 본 아이템 j 보다 선호 되어야 함
- ✓ $P(y_n = (u, i, j) | \theta) = \sigma(f(u, i; \theta) - f(u, j; \theta))$
- ✓ $\mathcal{L}(\theta) = \sum_{(u, i, j) \in D} \log \sigma(f(u, i; \theta) - f(u, j; \theta)) - \lambda \|\theta\|^2$ 을 최대화
- ✓ $\mathcal{D} = \{(u, i, j) : i \in \mathcal{I}_u^+, j \in \mathcal{I} \setminus \mathcal{I}_u^+\}$ $\mathcal{I} = \{i_1, i_2, i_3, i_4\}$ $\mathcal{I}_u^+ = \{i_2, i_3\}$
- ✓ $Y_{u, i_2, i_3} = ? \rightarrow$ 둘 다 positive
- ✓ $Y_{u, i_4, i_1} = ? \rightarrow$ 둘 다 negative 또는 unknown

$$\mathbf{Y}_u = \begin{pmatrix} . & + & + & ? \\ - & . & ? & - \\ - & ? & . & - \\ ? & + & + & . \end{pmatrix}$$

05 | Recommender Systems

Factorization Machines

- Factorization Machines
 - ✓ 앞서 보았던 Matrix Factorization은 $\text{user} \times \text{item}$ 상호작용만 봄
 - ✓ Factorization Machine은 이것을 확장해서 모든 feature들 사이의 상호작용을 봄
 - ✓ 임의의 주어진 사용자-아이템 쌍 $x = [\text{one} - \text{hot}(u), \text{one} - \text{hot}(i)]$ 를 위한 출력을 다음의 식을 사용해 예측
 - ✓ $f(x) = \mu + \sum_{i=1}^D w_i x_i + \sum_{i=1}^D \sum_{j=i+1}^D (v_i^T v_j) x_i x_j \rightarrow$ 기본값 + 각 feature의 단독 효과 + feature들끼리의 상호작용 효과
 - ✓ $D = (M + N)$, $x \in \mathbb{R}^D$, $V \in \mathbb{R}^{D \times K}$ 가중치 행렬, $w \in \mathbb{R}^D$ 는 가중치 벡터, $\mu \in \mathbb{R}$ 은 전역적인 단차
- Why Factorization Machines?
 - ✓ 현실 추천시스템에는 정보가 훨씬 많음 Ex) 장르, 시간대, 기기, 사용자 나이, 지역, 가격 등
 - ✓ Factorization Machine은 feature들을 모두 하나의 입력 벡터로 표현하고, feature들 사이의 쌍별 상호작용을 학습
 - ✓ 즉, FM은 Matrix Factorization을 더 일반화한 추천 모델

Q&A

감사합니다.